КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА

Факультет інформаційних технологій
Кафедра програмних систем і технологій

ЗВІТ

з лабораторної роботи №4
з дисципліни «Об'єктно-орієнтоване програмування»

Тема: «Застосування об'єктно-орієнтованого підходу при створенні графічних інтерфейсів користувача»

Проєкт: «Автоматизована система “Склад запчастин для авто”»

Виконав: студент групи ІПЗ-21, підгрупа 1
Агапов Олександр Олексійович

Перевірила: доц. Зубик Л. В.

Київ - 2026

1. Мета та завдання роботи

Метою роботи є створення та завершення графічного інтерфейсу для автоматизованої системи обліку запчастин. У застосунку на C#, ASP.NET Core MVC та SQLite реалізовано операції додавання, перегляду, коригування й вилучення даних з урахуванням зв'язків між сутностями.

2. Архітектура графічного інтерфейсу

Користувач взаємодіє з Razor Views у браузері. Controller обробляє GET- або POST-запит, перевіряє модель та через WarehouseContext звертається до SQLite. Entity Framework Core виконує операцію, після чого результат відображається у View. Для форм зі складеними даними застосовано ViewModel.

Під час запуску застосунку migrations створюють або оновлюють базу, а SeedData додає початкові дані лише за відсутності записів.

```
context.Database.Migrate();
SeedData.Initialize(context);
```

3. Реалізація графічного інтерфейсу

Меню містить розділи запчастин, критичних залишків, замовлень, постачальників, моделей автомобілів і звітів. Дані показано в таблицях, а створення й редагування виконуються через Razor-форми з валідацією.

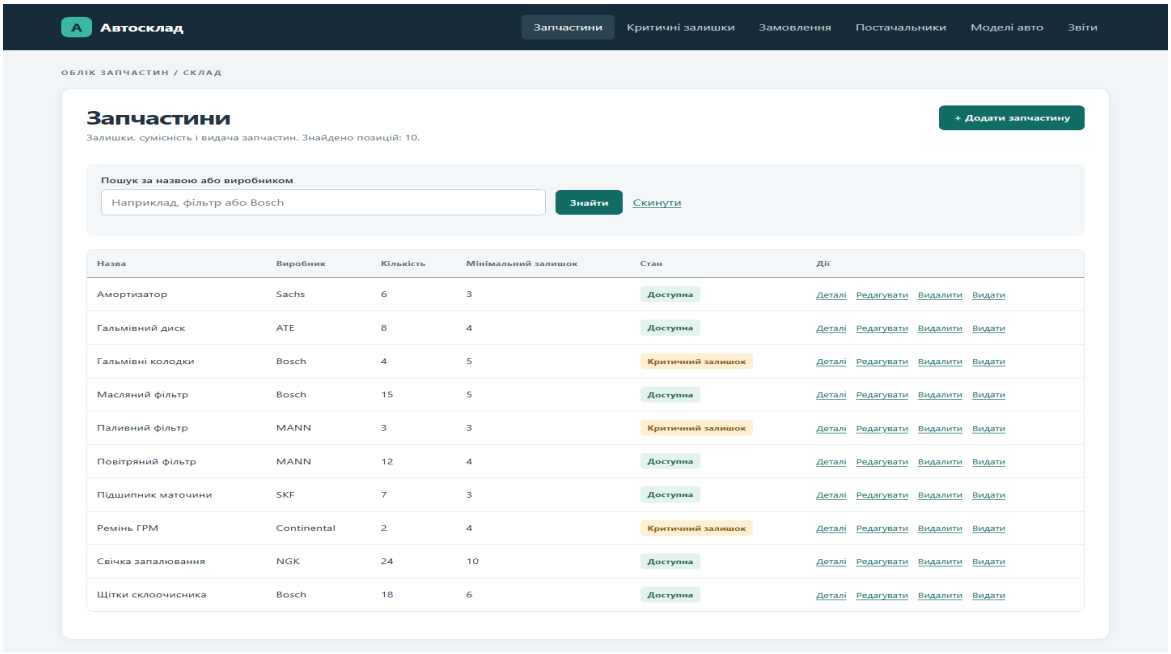


Рисунок 1 - Головна сторінка зі списком запчастин і навігацією.

4. Розширення CRUD-функціоналу

4.1. Постачальники

Для постачальників додано Edit і Delete. Перед видаленням перевіряється, чи є пов'язані замовлення; за їх наявності видалення блокується.

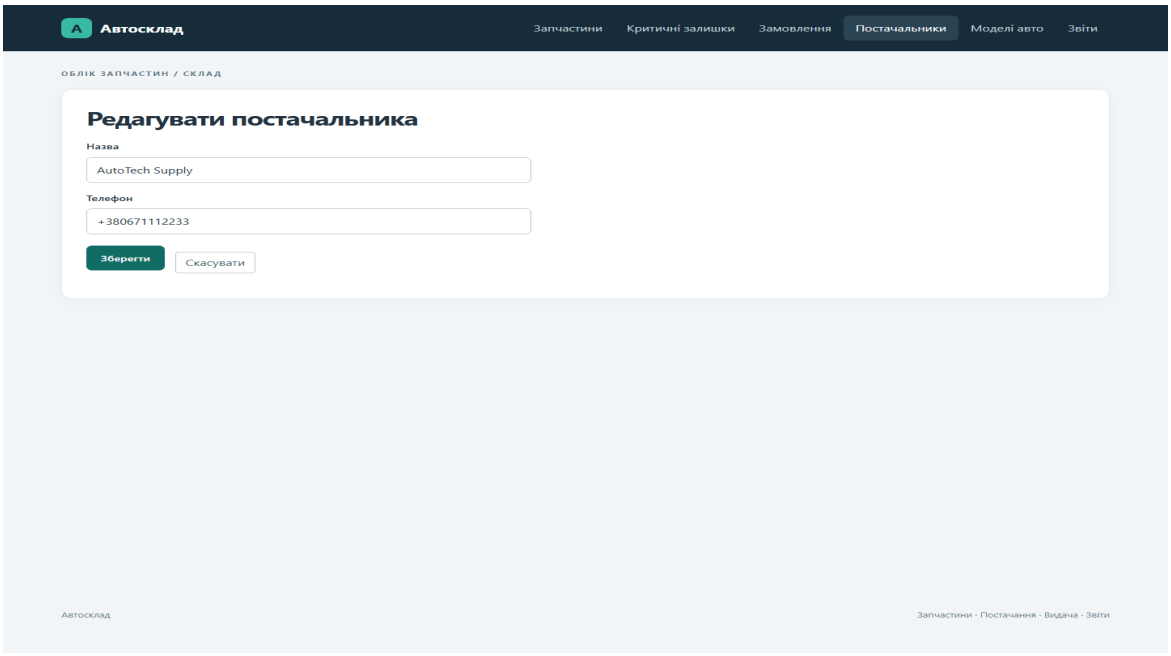


Рисунок 4 - Форма редагування даних постачальника.

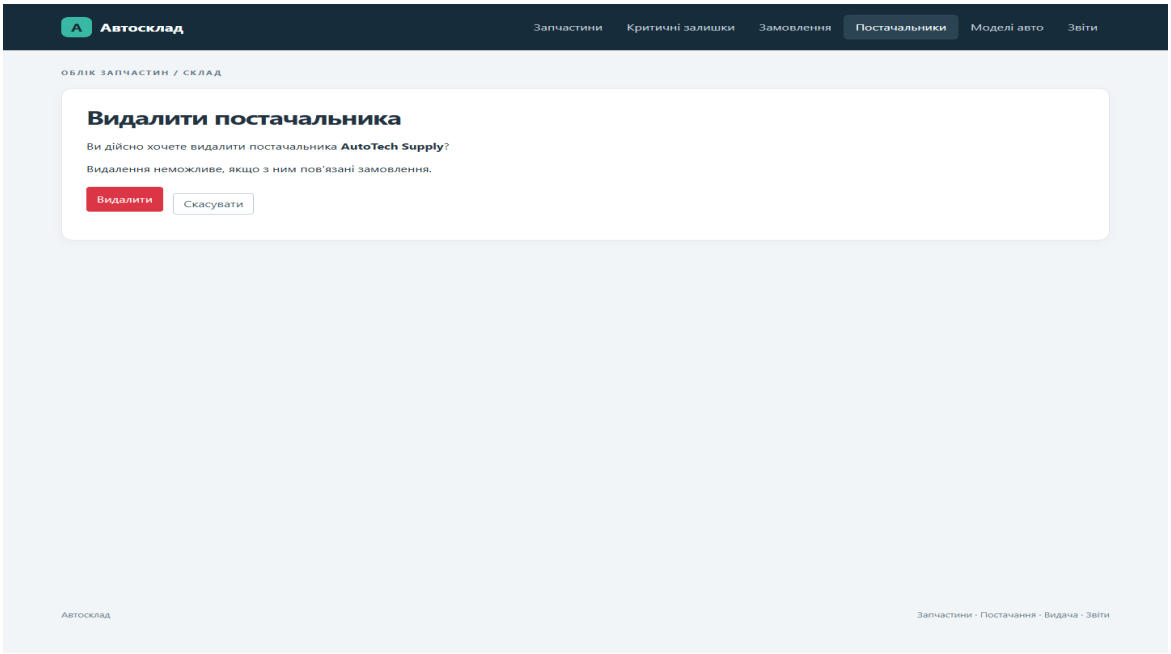


Рисунок 5 - Попередження перед видаленням постачальника, пов'язаного із замовленнями.

4.2. Моделі автомобілів

VehicleModelsController реалізує Create, Read, Update і Delete. VehicleModelViewModel передає дані форми та вибрані запчастини. Багато-до-багатьох зберігається через навігаційну колекцію AutoParts.

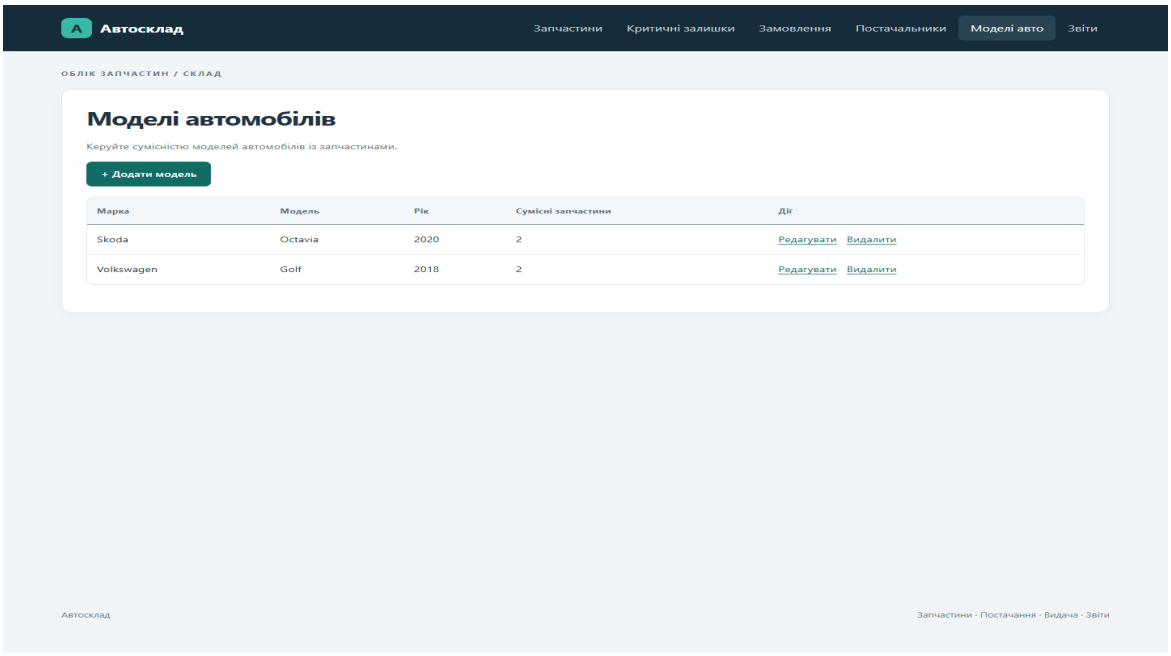


Рисунок 2 - Сторінка керування моделями автомобілів.

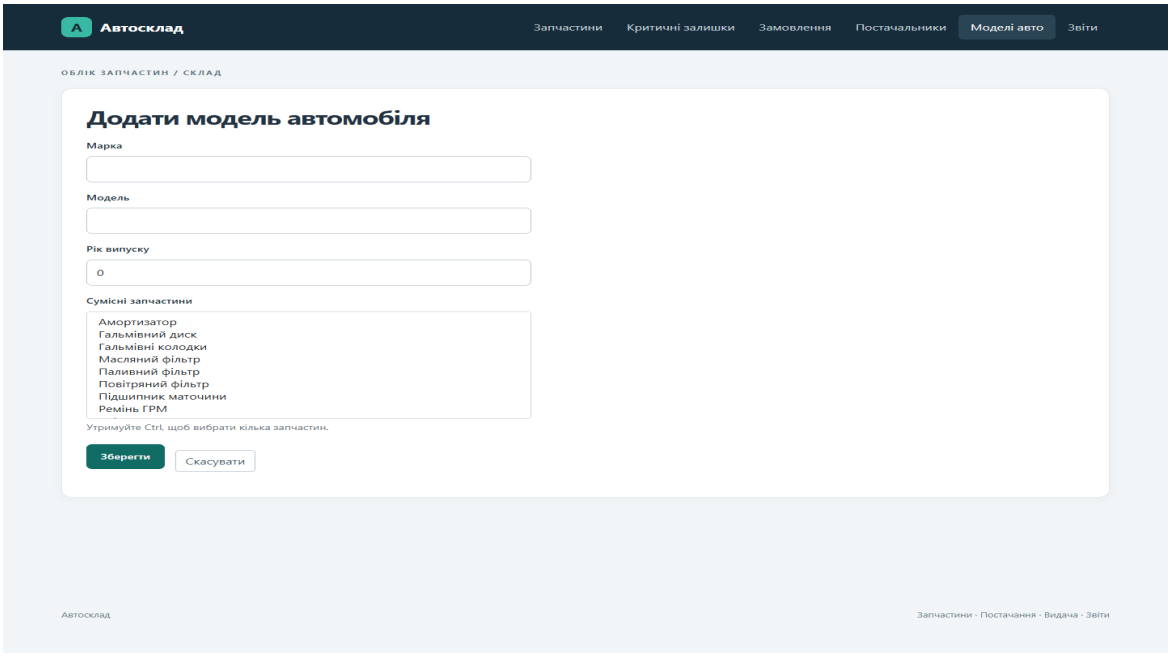


Рисунок 3 - Форма додавання моделі автомобіля з вибором сумісних запчастин.

4.3. Замовлення

Редагування та видалення дозволено лише для неотриманих замовлень. Позиції замовлення змінюються через форму самого замовлення. Після приймання зміна блокується, оскільки залишки вже оновлено.

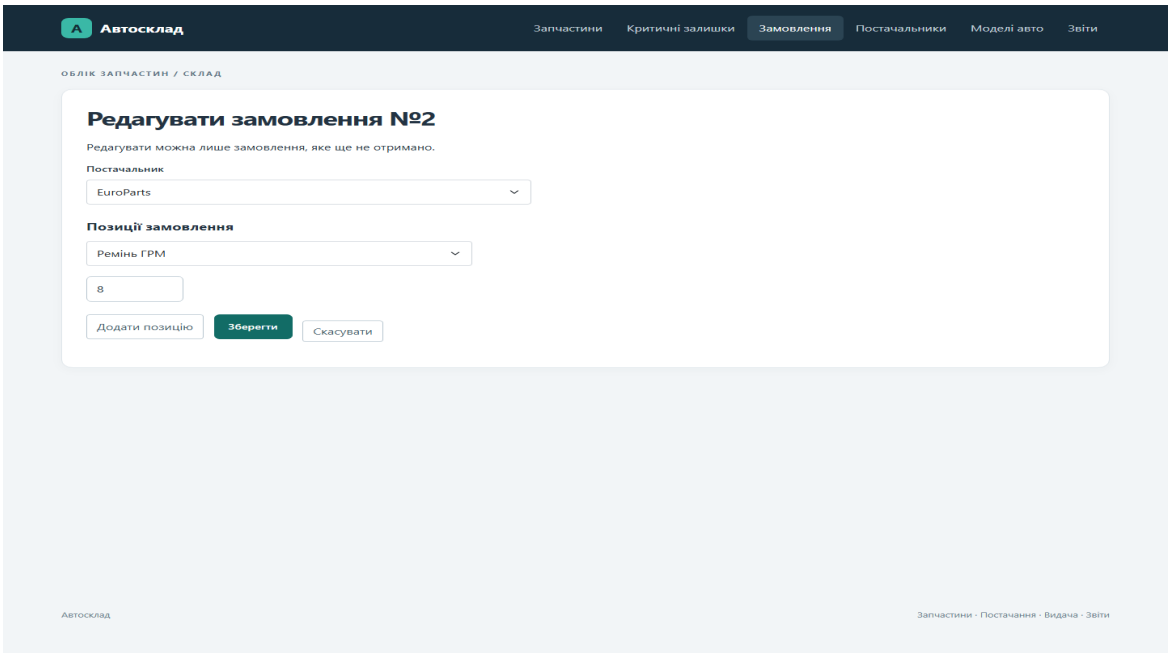


Рисунок 6 - Редагування неотриманого замовлення та його позицій.

У контролері ця умова перевіряється і під час відкриття форми, і перед POST-збереженням. Отже, обмеження не можна обійти простим повторним надсиланням застарілої форми.

```
if (order.Status == PurchaseOrderStatus.Received)
{
return RedirectToAction(nameof(Details), new { id });
}
```

4.4. Видача запчастин

Для PartIssue не використовується довільне редагування. Операція CancelIssue у транзакції повертає кількість на склад і видаляє запис видачі, тому дані залишаються узгодженими.

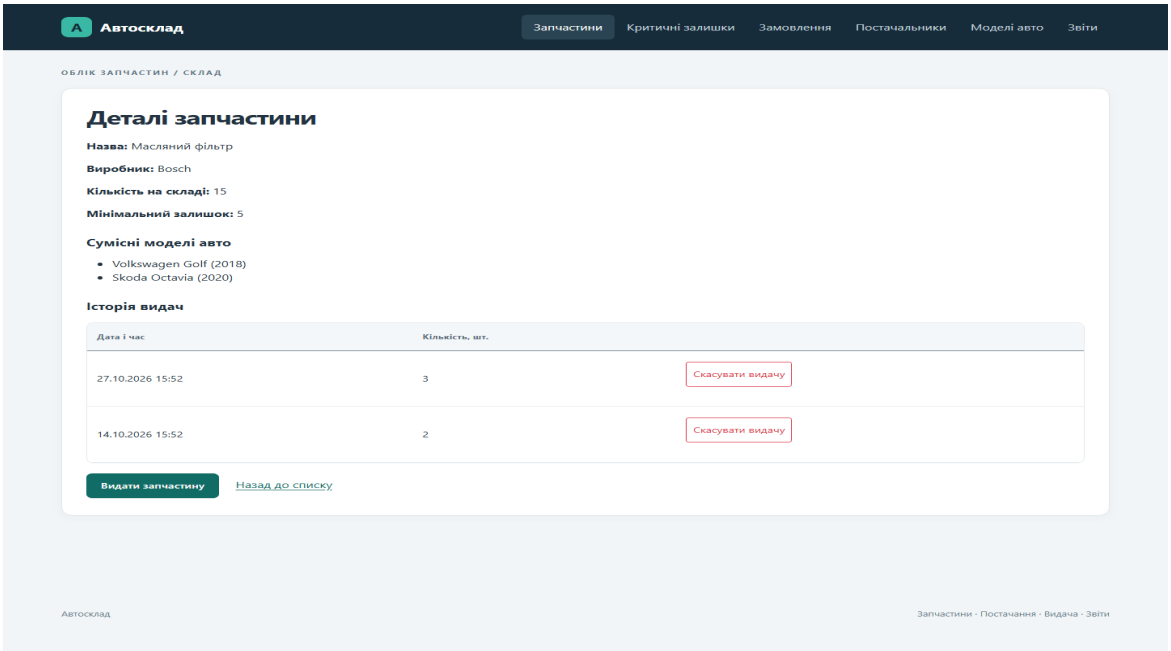


Рисунок 7 - Історія видач запчастини та безпечне скасування видачі.

5. Результат

Усі додані сторінки були перевірені на окремій тестовій SQLite-базі. Migrations і SeedData виконуються у правильному порядку, а повторний старт не створює дублікатів.

Сутність	C	R	U	D / обмеження
AutoPart	так	так	так	з історією не видаляється
Supplier	так	так	так	без замовлень
VehicleModel	так	так	так	так
PurchaseOrder	так	так	так	до приймання
PartIssue	видача	так	ні	скасування у транзакції

Таблиця 1 - Підтримувані операції для сутностей системи.

6. Висновок

У лабораторній роботі реалізовано цілісний вебінтерфейс для керування складом запчастин. Додані CRUD-операції та обмеження зберігають коректність зв'язків і складських залишків. Обидва завдання ЛР4 виконано.

Використані джерела

1. Методичні матеріали з дисципліни «Об'єктно-орієнтоване програмування». Лабораторна робота №4.

2. Вихідний код навчального проєкту AutoPartsWarehouseLab4.